

Laboratorio 08 · Usuarios, roles, restricciones y límites de recursos

Duración: 35 minutos · Carpeta de trabajo: entorno/

Objetivos

Vas a diseñar los usuarios de la aplicación "tienda" aplicando el principio de mínimo privilegio: roles predefinidos, un rol personalizado, restricciones por origen de conexión y límites de tiempo de ejecución. También verás qué guarda realmente MongoDB cuando crea un usuario con SCRAM.

Preparación

```
cd entorno
docker compose up -d
docker compose exec toolbox bash /scripts/estado.sh
```

Si el replica set no existe (por ejemplo, porque lo borraste), reconstrúyelo con el atajo del laboratorio 05. Después entra en el toolbox:

```
docker compose exec toolbox bash
mongosh "$RS"
```

Parte A · Roles predefinidos

```
> use admin
> db.getRoles({ showBuiltinRoles: true }).map(r => r.role)
> db.getRole("readWrite", { showPrivileges: true
}).privileges[0].actions
> db.getRole("clusterMonitor", { showPrivileges: true
}).privileges.length
```

Parte B · Usuarios de la aplicación

Vamos a centralizar todos los usuarios en la base de datos `admin`, que es la práctica recomendada: así hay un único lugar donde auditar quién existe.

```
> use admin
> db.createUser({ user: "app_tienda", pwd: "AppTienda2026", roles: [ {
  role: "readWrite", db: "tienda" } ] })
> db.createUser({ user: "analista",   pwd: "Analista2026",   roles: [ {
  role: "read", db: "tienda" } ] })
> db.createUser({ user: "monitor",    pwd: "Monitor2026",    roles: [ {
  role: "clusterMonitor", db: "admin" } ] })
```

Sal de `mongosh` y comprueba lo que puede hacer cada uno:

```
U="mongodb://analista:Analista2026@mongo1,mongo2,mongo3/tienda?
replicaSet=rs0&authSource=admin"
mongosh "$U" --quiet --eval "db.pedidos.countDocuments()"
mongosh "$U" --quiet --eval "db.pedidos.insertOne({ prueba: 1 })"
M="mongodb://monitor:Monitor2026@mongo1,mongo2,mongo3/?
replicaSet=rs0&authSource=admin"
mongosh "$M" --quiet --eval "rs.status().ok"
mongosh "$M" --quiet --eval
"db.getSiblingDB('tienda').pedidos.findOne()"
```

Parte C · Un rol personalizado

El equipo de atención al cliente necesita consultar pedidos y clientes y **cambiar el estado** de un pedido, pero nada más: ni insertar, ni borrar, ni tocar productos.

```
mongosh "$RS"
```

```
> use admin
> db.createRole({
  role: "atencionCliente",
  privileges: [
    { resource: { db: "tienda", collection: "pedidos" }, actions: [
      "find", "update" ] },
    { resource: { db: "tienda", collection: "clientes" }, actions: [
      "find" ] }
  ],
  roles: []
})
```

```
    })
  > db.createUser({ user: "atencion", pwd: "Atencion2026", roles: [ {
    role: "atencionCliente", db: "admin" } ] })
  > exit
```

Prueba el usuario:

```
A="mongodb://atencion:Atencion2026@mongo1,mongo2,mongo3/tienda?
replicaSet=rs0&authSource=admin"
mongosh "$A" --quiet --eval "db.pedidos.findOne({ estado: 'pendiente'
})"
mongosh "$A" --quiet --eval "db.pedidos.updateOne({ estado: 'pendiente'
}, { \$set: { estado: 'pagado' } })"
mongosh "$A" --quiet --eval "db.pedidos.deleteOne({ estado: 'cancelado'
})"
mongosh "$A" --quiet --eval "db.productos.findOne()"
```

Ejercicio: el equipo pide ahora poder **ver** también el catálogo de productos. Modifica el rol sin volver a crear el usuario.

▼ Ver solución

```
> use admin
> db.grantPrivilegesToRole("atencionCliente", [ { resource: { db:
"tienda", collection: "productos" }, actions: [ "find" ] } ])
```

El cambio afecta inmediatamente a todos los usuarios con ese rol, sin tocar los usuarios. Esa es la ventaja de gestionar permisos mediante roles y no usuario a usuario.

Parte D · Challenge/response: ¿qué se guarda de una contraseña?

```
> use admin
> db.system.users.findOne({ user: "atencion" }, { credentials: 1 })
```

Fíjate en que hay dos mecanismos (`SCRAM-SHA-1` y `SCRAM-SHA-256`) y en que ninguno contiene la contraseña: solo una sal (`salt`), un número de iteraciones y dos claves derivadas (`storedKey` y `serverKey`). Crea un usuario que solo admita SCRAM-SHA-256:

```
> db.createUser({ user: "solo256", pwd: "Solo256Curso2026", roles: [ {
role: "read", db: "tienda" } ], mechanisms: [ "SCRAM-SHA-256" ] })
```

```
> db.system.users.findOne({ user: "solo256" }, { credentials: 1 })
```

▼ Ver solución

En SCRAM la contraseña nunca viaja por la red ni se guarda en el servidor. El servidor envía la sal y el número de iteraciones; el cliente deriva una clave a partir de la contraseña y demuestra que la conoce firmando los mensajes del intercambio. El servidor comprueba esa prueba con `storedKey` y, a su vez, demuestra al cliente que también conoce el secreto usando `serverKey`: la autenticación es **mutua**. SCRAM-SHA-1 se mantiene por compatibilidad con drivers antiguos; si todos tus clientes son modernos, conviene usar solo SCRAM-SHA-256.

Parte E · Restricciones por origen

Crea un usuario de informes que solo pueda conectarse desde una red concreta que **no** es la nuestra:

```
> use admin
> db.createUser({
  user: "informes", pwd: "Informes2026",
  roles: [ { role: "read", db: "tienda" } ],
  authenticationRestrictions: [ { clientSource: [ "10.99.0.0/16" ] } ]
})
> exit
```

```
mongosh "mongodb://informes:Informes2026@mongo1,mongo2,mongo3/tienda?
replicaSet=rs0&authSource=admin" --quiet --eval
"db.pedidos.countDocuments()"
hostname -i
```

La autenticación falla aunque la contraseña es correcta. Ahora autoriza **solo la IP del toolbox** (la que te ha devuelto `hostname -i`):

```
> use admin
> db.updateUser("informes", { authenticationRestrictions: [ {
  clientSource: [ "172.18.0.5/32" ] } ] }) // pon tu IP
```

Vuelve a probar la conexión. Por último, busca en los logs el rastro del intento fallido (desde tu equipo):

```
docker compose logs mongo1 mongo2 mongo3 | grep -i "Authentication
failed" | tail -3
```

Parte F · Límites de recursos

F.1 Límite de tiempo por operación

```
> use tienda
> db.pedidos.aggregate([ { $unwind: "$lineas" }, { $group: { _id:
"$lineas.producto_id", n: { $sum: 1 } } } ], { $sort: { n: -1 } }
]).toArray().length
> db.pedidos.aggregate([ { $unwind: "$lineas" }, { $group: { _id:
"$lineas.producto_id", n: { $sum: 1 } } } ], { maxTimeMS: 10
}).toArray()
```

F.2 Límite por defecto para todas las lecturas (MongoDB 8.0)

Fija un límite por defecto de 20 ms para las lecturas:

```
> db.adminCommand({ setClusterParameter: { defaultMaxTimeMS: {
readOperations: 20 } } })
> db.adminCommand({ getClusterParameter: "defaultMaxTimeMS" })
> exit
```

Lanza la misma agregación con el usuario `analista` (el usuario `admin` tiene rol `root`, que puede saltarse este límite):

```
U="mongodb://analista:Analista2026@mongo1,mongo2,mongo3/tienda?
replicaSet=rs0&authSource=admin"
mongosh "$U" --quiet --eval 'db.pedidos.aggregate([ { $unwind: "$lineas"
}, { $group: { _id: "$lineas.producto_id", n: { $sum: 1 } } }
]).toArray().length'
```

Y deja el parámetro como estaba:

```
mongosh "$RS" --quiet --eval 'db.adminCommand({ setClusterParameter: {
defaultMaxTimeMS: { readOperations: 0 } } })'
mongosh "$RS"
```

F.3 Conexiones

```
> db.serverStatus().connections
> db.adminCommand({ getCmdLineOpts: 1 }).parsed.net
```

▼ Ver solución

Con `maxTimeMS: 10` la agregación se interrumpe con `MaxTimeMSExpired`: el servidor corta la operación y libera los recursos. Con `defaultMaxTimeMS` (nuevo en 8.0) se fija un límite por defecto para **todas** las lecturas que no traigan el suyo propio; es una red de seguridad frente a consultas descontroladas, pero hay que elegir el valor con cuidado para no romper informes legítimos (los usuarios con el privilegio `bypassDefaultMaxTimeMS`, como `root`, no se ven afectados). El número máximo de conexiones entrantes se controla con `net.maxIncomingConnections` y, en la práctica, también con los límites del sistema operativo (`ulimit -n`) y con el tamaño de los pools de conexión de los drivers, que es donde suele estar el problema real.

Limpieza

Deja los usuarios creados: los usaremos en el resto del día.